

100 Days of Linux - Day 093

Title: Accepting User Input with read

Today's Goal

Learn how to accept input from the user in Bash scripts using the **read** command.

Why This Matters

Interactive scripts are used in automation, deployments and administration to collect information before performing tasks.

Commands of the Day

Syntax:

```
read NAME
echo $NAME
read -p "Enter your name: " NAME
```

Example Script:

```
#!/bin/bash
read -p "Enter your name: " NAME
echo "Welcome, $NAME!"
```

Beginner Lesson

The **read** command pauses a script and waits for the user to type a value. The entered value is stored in a variable and can be used later in the script.

Practice Questions

1. Create a script named input.sh.
2. Ask the user for their name using read.
3. Ask for their city and store it in CITY.
4. Display a welcome message using both values.
5. Ask for their favourite programming language and print a complete sentence using all three inputs.

Hints

Use **read -p** to display a prompt. Access stored values with the dollar sign (\$).

Mini Challenge

Create a script that asks for your name, age and favourite Linux command, then displays a formatted summary.

Common Mistakes

Forgetting to use different variable names, missing the dollar sign when printing values or placing spaces around '=' when assigning variables.

Did You Know?

Many installation scripts begin by collecting information from the user with the **read** command.

Pro Tip

Write clear prompts so users know exactly what information to enter.

Answer Key (Instructor Only)

```
Q1: nano input.sh
Q2: read -p "Enter your name: " NAME
Q3: read -p "Enter your city: " CITY
Q4: echo "Welcome $NAME from $CITY"
Q5: read -p "Favourite language: " LANG then echo "$NAME from $CITY likes $LANG"
```

Homework

Build a script that collects five pieces of personal information and displays them in a neat summary.

Next Day Preview

Tomorrow you'll learn how to use command-line arguments in Bash scripts.